



Grafické uživatelské rozhraní v režimu okamžitého vykreslování

Bc. Radek Mocek

Ing. Jiří Jeníček, Ph.D.

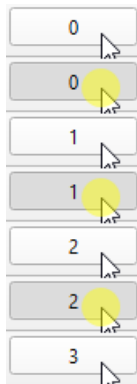
Cíle diplomové práce

1. Proved'te rešerši GUI v režimu okamžitého vykreslování.
2. Navrhněte ukázkovou aplikaci.
3. Ve vybraných knihovnách implementujte a otestujte aplikaci.
4. Zhodnoťte výsledky.

RMGUI – Retained Mode GUI

- „Režim soběstačného vykreslování“
- V inicializační fázi se definují jednotlivé ovládací prvky a na ně se navěšují callback metody
- Hlavní smyčka pak řeší vykreslování prvků a reakce na události, před programátorem je skryta
- Knihovny *Qt*, *GTK*, *wxWidgets*, *WPF*, ...

```
fn init():  
    m_value = 0  
    m_button = new GUI::Btn(str(m_value))  
    layout.add_widget(m_button)  
    GUI::connect(m_button, GUI::Btn::click, button_click)  
  
fn button_click():  
    m_value += 1  
    m_button.set_text(str(m_value))
```



IMGUI – Immediate mode GUI

- „Režim okamžitého vykreslování“
- Žádná inicializační fáze ani callback metody, pouze hlavní smyčka
- Knihovny *Dear ImGui*, *egui*, *Nuklear*, ...

```
fn update():  
    static value = 0  
    if GUI::button(str(value)):  
        value += 1
```

- Definice zmiňuje pouze tento rozdíl z pohledu programátora koncové aplikace, implementace knihoven mohou být různé.
- V praxi ale:
 - Využití grafického API pro vykreslování ovládacích prvků (*OpenGL*, *DirectX*, ...)
 - Časté použití pro přidání ovládacích prvků do existující interaktivní aplikace
 - Spíše GUI pro ladění vývojářem, než *user-facing GUI*

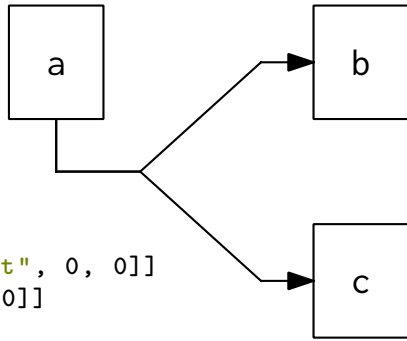
Motivace

- Vede na kratší kód a ImGui knihovny jsou v aktivním vývoji
- Jak jsou ImGui knihovny použitelné pro psaní běžných user-facing aplikací?
- Jaký je dopad na výkon oproti RMGUI knihovnám?

Návrh ukázkové aplikace

- Program, který vykreslí diagram na základě dodaného strukturovaného popisu
 - Kromě základních ovládacích prvků prozkoumá víceřádkové textové pole a schopnost vykreslování vlastních grafických primitiv

```
[node.a]
[node.b]
xy = ["a", "top-right", 100, 0]
[node.c]
pivot = "top"
xy = ["b", "bottom", 0, 50]
[[path]]
start = ["a", "bottom", 0, 0]
ends = [["b", "left", 0, 0], ["c", "left", 0, 0]]
points = [["", "prev", 40, "", "prev", 0]]
shift = 25
```



Ukázková aplikace

File View Debug Help

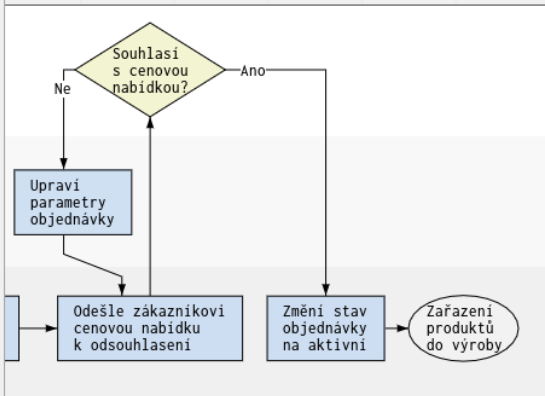
Font size: 20 - + Cursor pos: 22,38 Node color:  Type: ☐ Ellipse ▼ ID: s7

```
[node.bg_bg]
z = 2
size = [1602, 420]

[node.bg_z]
z = 3
value = "Zákazník"
size = ["bg_width", "bg_height"]
label_pos = "top-left"
xy = ["bg_bg", "top-left", 1, 1]
color = "#ffffff"
color_border = "#ffffff00"
label_shift = ["bg_label_shift", "abcd"]

[node.bg_p]
z = 3
value = "Pracovník"
size = ["bg_width", "bg_height"]
label_pos = "top-left"
pivot = "top"
```

Variable 'abcd' does not exist

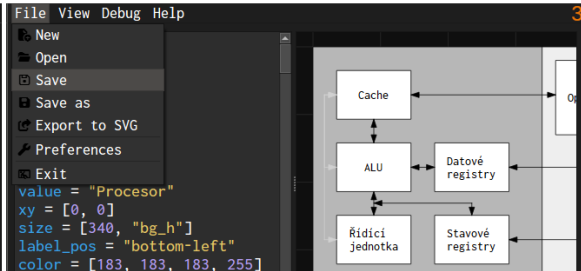
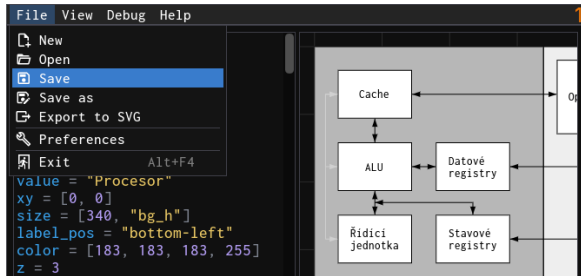


```
graph TD
    Start([Start]) --> SendOffer[Odešle zákazníkovi cenovou nabídku k odsouhlasení]
    SendOffer --> AdjustParams[Upravi parametry objednávky]
    AdjustParams --> Decision{Souhlasí s cenovou nabídkou?}
    Decision -- Ano --> ChangeStatus[Změni stav objednávky na aktivní]
    ChangeStatus --> AssignProd((Zařazení produktů do výroby))
    Decision -- Ne --> AdjustParams
```

□ ○ ◇ A Zoom level: 0.78

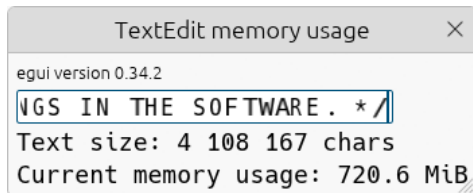
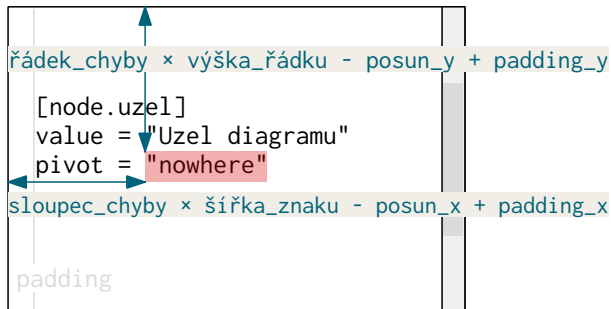
Ukázková aplikace

- Implementována v:
 1. Dear ImGui (C++, IMGUI)
 2. egui (Rust, IMGUI)
 3. Qt (C++, RMGUI)
- Podporuje:
 - Zvýrazňování syntaxe
 - Tmavý režim
 - Nativní dialogy pro uložení/nahrání
 - Export diagramu do formátu SVG



ImGui a textová pole

- Textové pole je komplexní ovládací prvek, který má napříč ImGui knihovnami omezené funkce
- V knihovnách *Dear ImGui* a *egui*:
 - Pro zvýrazňování syntaxe používám doplněk třetí strany
 - Pro vyznačení chyby vykresluji červený obdélník nad textovým polem
- Výchozí textové pole v knihovně Qt toto podporuje
- Textové pole v *egui* má problém s nadměrným využitím operační paměti



Programování v knihovně Qt

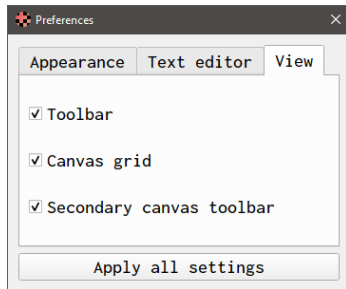
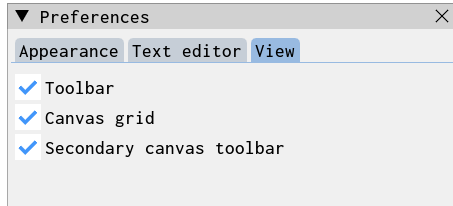
- Založeno na callback metodách a signálech

```
connect(m_scene, &Scene::EmptySpaceClicked, this, &MainWindow::OnEmptySpaceClick);  
connect(m_scene, &Scene::NodeClicked, this, &MainWindow::OnNodeClick);  
connect(m_scene, &Scene::NodeCtrlClicked, this, &MainWindow::OnNodeCtrlClick);  
connect(m_scene, &Scene::NodeHoverEntered, this, &MainWindow::OnNodeHoverEnter);  
connect(m_scene, &Scene::NodeHoverLeft, this, &MainWindow::OnNodeHoverLeave);  
connect(m_scene, &Scene::GhostNodePlaced, this, &MainWindow::OnGhostNodePlace);
```

- Definice související logiky často na třech různých místech
- S přibývajícími funkcemi se zvyšuje rozštěpení logiky v aplikačním kódu
- Knihovna může interně spouštět callback metody v nečekaných situacích
 - Např. zapnutí zvýrazňování syntaxe se bere jako změna obsahu dokumentu
 - $\implies$ Nečekané chování, horší ladění
- Některé poskytnuté metody se nechovají konzistentně
 - Např. pokud uživatel táhne tlačítko, přestane se volat `mouseMoveEvent`
 - $\implies$ Některé problémy je nutné řešit na nižší úrovni abstrakce

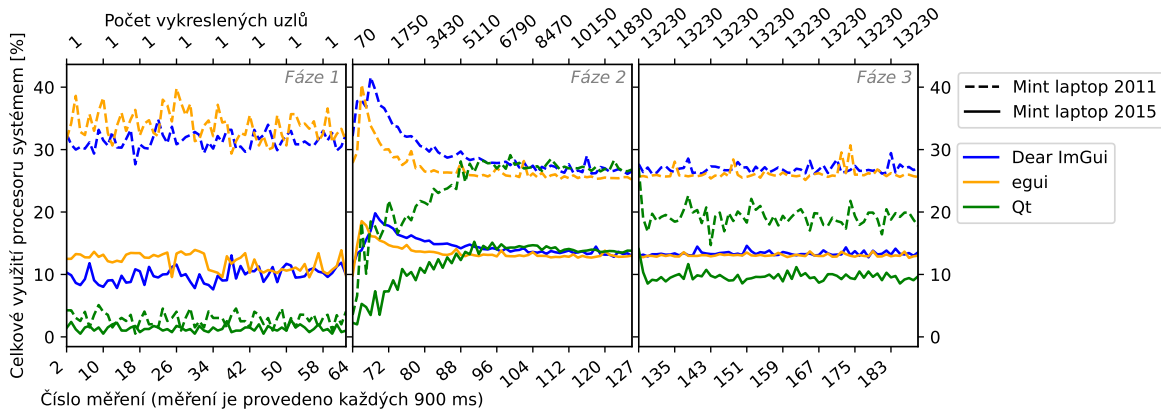
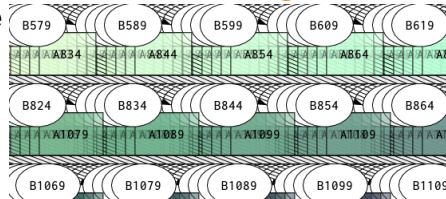
Dear ImGui a Qt: délka kódu

```
// --- Dear ImGui ---  
if (ImGui::BeginTabBar("PreferencesTabBar")) {  
    if (ImGui::BeginTabItem("View")) {  
        ImGui::Checkbox("Toolbar", &m_do_show_toolbar);  
        ImGui::Checkbox("Canvas grid", &m_do_show_grid);  
        ImGui::Checkbox("Secondary canvas toolbar", &m_do_show_sct);  
        ImGui::EndTabItem();  
    }  
    ImGui::EndTabBar();  
}  
  
// --- Qt ---  
const QPointer layout = new QVBoxLayout();  
const QPointer tabs = new QTabWidget();  
const QPointer layout_view_wrapper = new QWidget();  
const QPointer layout_view = new QVBoxLayout(layout_view_wrapper);  
m_view_toolbar = new QCheckBox("Toolbar");  
m_view_toolbar->setChecked(m_state.do_show_toolbar);  
layout_view->addWidget(m_view_toolbar);  
m_view_canvas_grid = new QCheckBox("Canvas grid");  
m_view_canvas_grid->setChecked(m_state.do_show_canvas_grid);  
layout_view->addWidget(m_view_canvas_grid);  
m_view_secondary_toolbar = new QCheckBox("Secondary canvas toolbar");  
m_view_secondary_toolbar->setChecked(m_state.do_show_sct);  
layout_view->addWidget(m_view_secondary_toolbar);  
tabs->addTab(layout_view_wrapper, "View");  
layout->addWidget(tabs);  
const QPointer button_apply = new QPushButton("Apply all settings");  
layout->addWidget(button_apply);  
connect(button_apply, &QPushButton::clicked, [this] { ... });  
setLayout(layout);
```



Porovnání výkonu aplikací: velké množství uzlů v diagramu

- Benchmark má tři fáze: jeden uzel ve scéně¹, postupné přidávání uzlů², velké množství uzlů ve scéně³
- Knihovny s ImGui mají vyšší počáteční náklady, knihovna Qt je ve druhé fázi dožene
- Stejný trend na různých zařízeních



Závěr

V rámci práce jsem provedl:

- Rešerši dvou přístupů ke psaní GUI aplikací: RMGUI a IMGUI
- Rešerši nejčastěji používaných IMGUI knihoven
- Návrh a implementaci ukázkových aplikací: editor diagramů v knihovnách *Dear ImGui*, *egui* a *Qt*
- Porovnání výsledných aplikací:
 - Z pohledu aplikačního kódu
 - Z pohledu koncového uživatele
 - Ve dvou typech benchmarku (velký počet uzlů v diagramu a velký počet ovládacích prvků v okně)



Děkuji za pozornost

„Jedna z implementací ukázkového programu byla v jazyce Rust. Jsou v přístupu k tvorbě GUI v Rustu významné rozdíly oproti jiným programovacím jazykům? Uvítal byste výuku Rustu na univerzitě?“

- Záleží na konkrétní knihovně, ale pro definici GUI je častá kompozice struktur, nebo *builder pattern*:

```
egui::TextEdit::multiline(&mut source).desired_width(f32::INFINITY)
    .code_editor().layouter(&mut layouter).show(ui);
```

- Knihovna egui kombinuje builder pattern s vnořováním uzávěrů. Obecný přístup ke tvorbě aplikace je zde podobný jako v Dear ImGui.
- Pro potřeby IMGUI chybí v jazyce Rust klíčové slovo `static`, takže všechna persistentní data musí být součástí nějaké struktury
- *The roots aren't deep but the seeds are planted.*
— <https://areweguiyet.com/>

„Pátral jste po důvodu, proč egui spotřebovává tolik paměti v případě použití textového pole?“

- **Knihovna připravuje na vykreslení veškerý text bez ohledu na to, jestli je aktuálně vidět**
 - Je možné přijít s vlastní logikou, která vykresluje jen to potřebné, ale pouze pro read-only víceřádková textová pole
- Spotřeba se navíc může několikanásobně zvýšit při náhodné editaci textu uživatelem, oproti Dear ImGui je zde i vyšší spotřeba při vykreslení velkého množství ovládacích prvků (i těch bez textu)
 - Autor knihovny se k tomu oficiálně nevyjádřil, ale vypadá to spíše na „důsledek robustní implementace“, která tento problém nebere v potaz, nežli konkrétní chybu v kódu, která by šla snadno opravit

AAAAAAAAAAAAAAAAAAAAAAAAAAAA

Text size: 1 793 988 chars

Current memory usage: 8054.8

Main update loop 10113.963 ms

TextEdit Window show 10113.805 ms

text_edit_singleline call 10113.692 ms

„Jak je to s použitím IMGUI a RMGUI přístupů na mobilních platformách?“

- Obecně (nejen na mobilu) se nejedná o černobílou problematiku: RMGUI vs. IMGUI je zjednodušený pohled
- Populární jsou JavaScript knihovny, kde programátor definuje *komponenty*, ze kterých se skládá GUI
 - Správa stromu komponent je *soběstačná*, definice komponent mohou připomínat *okamžitý* režim („locality of behavior“, možnost skrýt část GUI podmíněním, ...)
- Nejčastěji používané knihovny pro tvorbu mobilních GUI také využívají přístupů s komponentami (Jetpack Compose, Flutter, React Native, ...)
- Knihovny použité v této práci podporují platformy Android/iOS, ale nejsou v tomto ohledu zdaleka tak populární jako výše zmíněné